

# RISC-V 32 Reference Card

## General Purpose Registers

| Register | ABI Name | Description    | Saver  | Register | ABI Name | Description           | Saver  |
|----------|----------|----------------|--------|----------|----------|-----------------------|--------|
| x0       | zero     | Zero constant  | —      | x8       | s0 / fp  | Saved / frame pointer | Callee |
| x1       | ra       | Return address | Caller | x9       | s1       | Saved register        | Callee |
| x2       | sp       | Stack pointer  | Callee | x10-x11  | a0-a1    | Fn args/return values | Caller |
| x3       | gp       | Global pointer | —      | x12-x17  | a2-a7    | Fn args               | Caller |
| x4       | tp       | Thread pointer | —      | x18-x27  | s2-s11   | Saved registers       | Callee |
| x5-x7    | t0-t2    | Temporaries    | Caller | x28-x31  | t3-t6    | Temporaries           | Caller |

## RV32I Base Integer Instructions

| Inst             | Name                    | Description (C)              | Note                                      |
|------------------|-------------------------|------------------------------|-------------------------------------------|
| add rd,rs1,rs2   | ADD                     | rd = rs1 + rs2               | msb-ext<br>signed<br>unsigned             |
| sub rd,rs1,rs2   | SUB                     | rd = rs1 - rs2               |                                           |
| xor rd,rs1,rs2   | XOR                     | rd = rs1 ^ rs2               |                                           |
| or rd,rs1,rs2    | OR                      | rd = rs1   rs2               |                                           |
| and rd,rs1,rs2   | AND                     | rd = rs1 & rs2               |                                           |
| sll rd,rs1,rs2   | Shift Left Logical      | rd = rs1 << rs2              |                                           |
| srl rd,rs1,rs2   | Shift Right Logical     | rd = rs1 >> rs2              | zero-ext<br>msb-ext<br>signed<br>unsigned |
| sra rd,rs1,rs2   | Shift Right Arith       | rd = rs1 >> rs2              |                                           |
| slt rd,rs1,rs2   | Set Less Than           | rd = (rs1 < rs2)?1:0         |                                           |
| sltu rd,rs1,rs2  | Set Less Than (U)       | rd = (rs1 < rs2)?1:0         |                                           |
| addi rd,rs1,imm  | ADD (Immediate)         | rd = rs1 + imm               |                                           |
| xori rd,rs1,imm  | XOR (Immediate)         | rd = rs1 ^ imm               |                                           |
| ori rd,rs1,imm   | OR (Immediate)          | rd = rs1   imm               | zero-ext<br>msb-ext<br>signed<br>unsigned |
| andi rd,rs1,imm  | AND (Immediate)         | rd = rs1 & imm               |                                           |
| slli rd,rs1,imm  | Shift Left Logical Imm  | rd = rs1 << imm[0:4]         |                                           |
| srlr rd,rs1,imm  | Shift Right Logical Imm | rd = rs1 >> imm[0:4]         |                                           |
| srai rd,rs1,imm  | Shift Right Arith Imm   | rd = rs1 >> imm[0:4]         |                                           |
| slti rd,rs1,imm  | Set Less Than Imm       | rd = (rs1 < imm)?1:0         |                                           |
| sltiu rd,rs1,imm | Set Less Than Imm (U)   | rd = (rs1 < imm)?1:0         | zero-ext<br>zero-ext                      |
| lb rd,imm(rs1)   | Load Byte               | rd = M[rs1+imm][0:7]         |                                           |
| lh rd,imm(rs1)   | Load Half               | rd = M[rs1+imm][0:15]        |                                           |
| lw rd,imm(rs1)   | Load Word               | rd = M[rs1+imm][0:31]        |                                           |
| lbu rd,imm(rs1)  | Load Byte (U)           | rd = M[rs1+imm][0:7]         |                                           |
| lhu rd,imm(rs1)  | Load Half (U)           | rd = M[rs1+imm][0:15]        |                                           |
| sb rs2,imm(rs1)  | Store Byte              | M[rs1+imm][0:7] = rs2[0:7]   |                                           |
| sh rs2,imm(rs1)  | Store Half              | M[rs1+imm][0:15] = rs2[0:15] |                                           |
| sw rs2,imm(rs1)  | Store Word              | M[rs1+imm][0:31] = rs2[0:31] | signed<br>signed<br>unsigned<br>unsigned  |
| beq rs1,rs2,imm  | Branch ==               | if(rs1 == rs2) PC += imm     |                                           |
| bne rs1,rs2,imm  | Branch !=               | if(rs1 != rs2) PC += imm     |                                           |
| blt rs1,rs2,imm  | Branch <                | if(rs1 < rs2) PC += imm      |                                           |
| bge rs1,rs2,imm  | Branch ≥                | if(rs1 ≥ rs2) PC += imm      |                                           |
| bltu rs1,rs2,imm | Branch < (U)            | if(rs1 < rs2) PC += imm      |                                           |
| bgeu rs1,rs2,imm | Branch ≥ (U)            | if(rs1 ≥ rs2) PC += imm      |                                           |
| jal rd,imm       | Jump And Link           | rd = PC+4; PC += imm         |                                           |
| jalr rd,rs1,imm  | Jump And Link Reg       | rd = PC+4; PC = rs1 + imm    |                                           |
| lui rd,imm       | Load Upper Imm          | rd = imm << 12               |                                           |
| auipc rd,imm     | Add Upper Imm to PC     | rd = PC + (imm << 12)        |                                           |

Adapted from: [github.com/jameszhu/riscv-card](https://github.com/jameszhu/riscv-card) (CC-BY-4.0)

## RV32M Multiply Extension

| Inst              | Name             | Description (C)         |
|-------------------|------------------|-------------------------|
| mul rd,rs1,rs2    | MUL              | rd = (rs1 * rs2)[31:0]  |
| mulh rd,rs1,rs2   | MUL High         | rd = (rs1 * rs2)[63:32] |
| mulhsu rd,rs1,rs2 | MUL High (S) (U) | rd = (rs1 * rs2)[63:32] |
| mulhu rd,rs1,rs2  | MUL High (U)     | rd = (rs1 * rs2)[63:32] |
| div rd,rs1,rs2    | DIV              | rd = rs1 / rs2          |
| divu rd,rs1,rs2   | DIV (U)          | rd = rs1 / rs2          |
| rem rd,rs1,rs2    | Remainder        | rd = rs1 % rs2          |
| remu rd,rs1,rs2   | Remainder (U)    | rd = rs1 % rs2          |

## Pseudo Instructions

| Pseudoinstruction         | Base Instruction(s)                                        | Meaning                       |
|---------------------------|------------------------------------------------------------|-------------------------------|
| la rd, symbol             | auipc rd, symbol[31:12]<br>addi rd, rd, symbol[11:0]       | Load address                  |
| l{b h w d} rd, symbol     | auipc rd, symbol[31:12]<br>l{b h w d} rd, symbol[11:0](rd) | Load global                   |
| s{b h w d} rd, symbol, rt | auipc rt, symbol[31:12]<br>s{b h w d} rd, symbol[11:0](rt) | Store global                  |
| fl{w d} rd, symbol, rt    | auipc rt, symbol[31:12]<br>fl{w d} rd, symbol[11:0](rt)    | Floating-point load global    |
| fs{w d} rd, symbol, rt    | auipc rt, symbol[31:12]<br>fs{w d} rd, symbol[11:0](rt)    | Floating-point store global   |
| nop                       | addi x0, x0, 0                                             | No operation                  |
| li rd, immediate          | <i>Myriad sequences</i>                                    | Load immediate                |
| mv rd, rs                 | addi rd, rs, 0                                             | Copy register                 |
| not rd, rs                | xori rd, rs, -1                                            | One's complement              |
| neg rd, rs                | sub rd, x0, rs                                             | Two's complement              |
| negw rd, rs               | subw rd, x0, rs                                            | Two's complement word         |
| sext.w rd, rs             | addiw rd, rs, 0                                            | Sign extend word              |
| seqz rd, rs               | sltiu rd, rs, 1                                            | Set if = zero                 |
| snez rd, rs               | sltu rd, x0, rs                                            | Set if ≠ zero                 |
| sltz rd, rs               | slt rd, rs, x0                                             | Set if < zero                 |
| sgtz rd, rs               | slt rd, x0, rs                                             | Set if > zero                 |
| beqz rs, offset           | beq rs, x0, offset                                         | Branch if = zero              |
| bnez rs, offset           | bne rs, x0, offset                                         | Branch if ≠ zero              |
| blez rs, offset           | bge x0, rs, offset                                         | Branch if ≤ zero              |
| bgez rs, offset           | bge rs, x0, offset                                         | Branch if ≥ zero              |
| bltz rs, offset           | blt rs, x0, offset                                         | Branch if < zero              |
| bgtz rs, offset           | blt x0, rs, offset                                         | Branch if > zero              |
| bgt rs, rt, offset        | blt rt, rs, offset                                         | Branch if >                   |
| ble rs, rt, offset        | bge rt, rs, offset                                         | Branch if ≤                   |
| bgtu rs, rt, offset       | bltu rt, rs, offset                                        | Branch if >, unsigned         |
| bleu rs, rt, offset       | bgeu rt, rs, offset                                        | Branch if ≤, unsigned         |
| j offset                  | jal x0, offset                                             | Jump                          |
| jal offset                | jal x1, offset                                             | Jump and link                 |
| jr rs                     | jalr x0, rs, 0                                             | Jump register                 |
| jalr rs                   | jalr x1, rs, 0                                             | Jump and link register        |
| ret                       | jalr x0, x1, 0                                             | Return from subroutine        |
| call offset               | auipc x1, offset[31:12]<br>jalr x1, x1, offset[11:0]       | Call far-away subroutine      |
| tail offset               | auipc x6, offset[31:12]<br>jalr x0, x6, offset[11:0]       | Tail call far-away subroutine |